hexens

SMART CONTRACT AUDIT REPORT FOR BITDRIVE

03.01.2022

Contents

- Summary / 3
- Scope / 3
- Description / 4
- Test Cases / 5
- Weaknesses / 7
  - Possible Reentrancy / 7
  - No Timelock Mechanism Used / 8
  - Centralization Risk / 8
  - Misprints in the Contract / 9
  - Return Value Not Checked / 9
  - Emit an Event / 10
  - Missing Zero Address Checks / 10

Summary

During the audit our specialists found 3 medium risk and multiple low risk weaknesses.

Severity	Number of Findings
High	0
Medium	3
Low	4
Informational	0

Total: 7

Scope

The analyzed contracts are located in BSC Testnet, and are available via following links:
https://testnet.bscscan.com/address/0x51e766e4f51e303b14b88dE3a3b7DfBA496CE1bF
https://testnet.bscscan.com/address/0x912A5515093E29daCF5f9975A2ab3512B01c84ec
https://testnet.bscscan.com/address/0x164510aff8eb331a4f0dca2fc6a58ec3d26c2868
The fixed contracts are located in BSC Testnet, and are available via following links:
https://testnet.bscscan.com/address/0xd20e8548800f7abbf8bc8bc8e586e25b238ae77d
https://testnet.bscscan.com/address/0x912A5515093E29daCF5f9975A2ab3512B01c84ec
https://testnet.bscscan.com/address/0x51e766e4f51e303b14b88dE3a3b7DfBA496CE1bF

Description

The BitDriveFactory.sol and BitDriveRouter.sol contracts are based on the Uniswap protocol. They are incorporated into the project with the following major changes:

BitDriveFactory.sol

- The fee collector address is set by default to feeToSetter in the contract constructor
- UniswapV2Pair name changed to SwapPair
- pairCodeHash() function added that returns SwapPair creator code hash
- Error messages changed

BitDriveRouter.sol

- SafeMath library name changed to SafeMathUniswap
- Error messages changed
- IERC20 renamed to IERC20Uniswap

The Masterchef.sol contract is based on the Goose protocol. It is incorporated into the project with the following major changes:

- MasterChef renamed to Masterchef
- Comment lines cleaned up
- accEggPerShare renamed to bidDrivePerShare
- EggToken renamed to Bitdrive

Test Cases

Integrated testing results are presented below.

```
Contract: BitDriveFactory
    ✓ feeTo, feeToSetter, allPairsLength (177ms)
    ✓ createPair (412ms)
    ✓ createPair:reverse (285ms)
    ✓ createPair:gas (156ms)
    ✓ setFeeTo (90ms)
    ✓ setFeeToSetter (100ms)

  UniswapV2Pair
    ✓ mint (252ms)
    ✓ getInputPrice:0 (298ms)
    ✓ getInputPrice:1 (271ms)
    ✓ getInputPrice:2 (272ms)
    ✓ getInputPrice:3 (275ms)
    ✓ getInputPrice:4 (295ms)
    ✓ getInputPrice:5 (261ms)
    ✓ getInputPrice:6 (258ms)
    ✓ optimistic:0 (289ms)
    ✓ optimistic:1 (257ms)
    ✓ optimistic:2 (275ms)
    ✓ optimistic:3 (246ms)
    ✓ swap:token0 (272ms)
    ✓ swap:token1 (276ms)
    ✓ swap:gas (304ms)
    ✓ burn (310ms)
    ✓ price{0,1}CumulativeLast (354ms)
    ✓ feeTo:off (310ms)
    ✓ feeTo:on (382ms)


  25 passing (8s)
```

```
Contract:BitDriveRouter
    ✓ quote (64ms)
    ✓ getAmountOut (50ms)
    ✓ getAmountIn (45ms)
    ✓ getAmountsOut (237ms)
    ✓ getAmountsIn (231ms)

  fee-on-transfer tokens
    ✓ removeLiquidityETHSupportingFeeOnTransferTokens (298ms)
    ✓
removeLiquidityETHWithPermitSupportingFeeOnTransferTokens
(326ms)
    ✓ swapExactETHForTokensSupportingFeeOnTransferTokens
(241ms)
    ✓ swapExactTokensForETHSupportingFeeOnTransferTokens
(196ms)
    swapExactTokensForTokensSupportingFeeOnTransferTokens
      ✓ DTT -> WETH (94ms)
      ✓ WETH -> DTT (120ms)

  fee-on-transfer tokens: reloaded
    swapExactTokensForTokensSupportingFeeOnTransferTokens
      ✓ DTT -> DTT2 (87ms)

  12 passing (7s)

Contract: MasterChef
    ✓ deposit fee (874ms)
    ✓ only dev (482ms)
    ✓ only fee (155ms)


  3 passing (6s)
```

Weaknesses

This section contains the list of discovered weaknesses.

1. Possible Reentrancy

Severity: Medium

Resolution: implement a reentrancy guard (e.g, inherit from OpenZeppelin ReentrancyGuard and add nonReentrant modifier)

Status: fixed

Description:

The **deposit()**, **withdraw()** and **emergencyWithdraw()** functions on lines 1491, 1524 and 1544 respectively in the file Masterchef.sol are not protected from reentrancy.

```
function deposit(uint256 _pid, uint256 _amount) public {
function withdraw(uint256 _pid, uint256 _amount) public {
function emergencyWithdraw(uint256 _pid) public {
```

2. No Timelock Mechanism Used

Severity: Medium

Resolution: Implement timelock functionality for Masterchef admin functionality

Status: fixed

Description:

There is no timelock mechanism implemented for the Masterchef administration.

3. Centralization Risk

Severity: Medium

Resolution: remove the safeBidDriveWithdraw() function

Status: fixed

Description:

There is a centralization risk in **safeBidDriveWithdraw()** function on line 1570 in the file Masterchef.sol, since the owner of the contract has the power to withdraw all of the token balance from the contract.

```
    function safeBidDriveWithdraw(address _addr, uint256 _amount)
        public
        onlyOwner
    {
        uint256 biddriveBal = biddrive.balanceOf(address(this));
        if (_amount > biddriveBal) {
            biddrive.transfer(_addr, biddriveBal);
        } else {
            biddrive.transfer(_addr, _amount);
        }
}
```

4. Misprints in the Contract

Severity: Low

Resolution: fix the misprints

Status: fixed

Description:

Throughout the contract the name **BitDrive** is misspelled and written as **BidDrive** instead. Also mentions of **BIZZPRO** token were found, which should be removed or renamed.

5. Return Value Not Checked

Severity: Low

Resolution: safeBidDriveTransfer() & safeBidDriveWithdraw()

should check transfer() return value

Status: fixed

Description:

The functions **safeBidDriveTransfer()** and **safeBidDriveWithdraw()** on lines 1555 and 1570 respectively in the file Masterchef.sol do not check **transfer()** function's return value.

```
function safeBidDriveTransfer(address _to, uint256 _amount)
function safeBidDriveWithdraw(address _addr, uint256 _amount)
```

6. Emit an Event

Severity: Low

Resolution: add event emit

Status: fixed

Description:

It is recommended to add event emits for **dev()**, **setFeeAddress()** and **updateEmissionRate()** functions on lines 1565, 1582 and 1588 respectively in the file Masterchef.sol.

```
function dev(address _devaddr) public
function setFeeAddress(address _feeAddress) public
function updateEmissionRate(uint256 _biddrivePerBlock) public
```

7. Missing Zero Address Checks

Severity: Low

Resolution: check for corresponding addresses not to be zero

Status: fixed

Description:

There are no zero address checks in the **dev()** and **setFeeAddress()** functions on lines 1565 and 1582 respectively in the file Masterchef.sol.

```
function dev(address _devaddr) public {
    require(msg.sender == devaddr, "dev: wut?");
    devaddr = _devaddr;
...
```

```
...
function setFeeAddress(address _feeAddress) public {
        require(msg.sender == feeAddress, "setFeeAddress:
FORBIDDEN");
        feeAddress = _feeAddress;
```

hexens